1. Nesne Yönelimli Programlamaya Giriş

Nesne Yönelimli Programlama (Object-Oriented Programming - OOP), yazılım geliştirme sürecinde gerçek dünya varlıklarını modellemeye odaklanır. Bu yaklaşım, yazılımı daha modüler, sürdürülebilir ve yeniden kullanılabilir hale getirir.

1.1 Temel Kavramlar

- **Sınıf (Class):** Nesnelerin şablonudur.
- **Nesne (Object):** Sınıfın bir örneğidir.
- **Kapsülleme (Encapsulation):** Verilerin ve metotların bir arada saklanması ve dış dünyadan gizlenmesi.
- **Kalıtım (Inheritance):** Bir sınıfın başka bir sınıfın özelliklerini devralması.
- **Polimorfizm (Polymorphism):** Aynı isimli metotların farklı şekillerde davranması.
- **Soyutlama (Abstraction):** Karmaşıklığı gizlemek ve önemli özellikleri öne çıkarmak.

---

2. Sınıf ve Nesne

2.1 Teorik Açıklama

Sınıf ve nesne, OOP'nin temel yapı taşlarıdır. Sınıflar bir tür "şablon" olarak tanımlanır ve bu şablondan nesneler oluşturulur.

2.2 Kod Örnekleri

Java

java

Kodu kopyala

```
class Araba {
   String marka;
   int yil;

   void calistir() {
      System.out.println(marka + " çalıştırıldı!");
   }
}
```

```
public class Main {
    public static void main(String[] args) {
        Araba araba = new Araba();
        araba.marka = "Toyota";
        araba.yil = 2020;
        araba.calistir();
    }
}
```

Python

```
python
Kodu kopyala
class Araba:
    def __init__(self, marka, yil):
        self.marka = marka
        self.yil = yil

    def calistir(self):
        print(f"{self.marka} çalıştırıldı!")

araba = Araba("Toyota", 2020)
araba.calistir()
```

C#

```
csharp
Kodu kopyala
class Araba {
    public string Marka { get; set; }
    public int Yil { get; set; }
```

```
    public void Calistir() {
        Console.WriteLine($"{Marka} çalıştırıldı!");
    }
}

class Program {
    static void Main(string[] args) {
        Araba araba = new Araba { Marka = "Toyota", Yil = 2020 };
        araba.Calistir();
    }
}
```

Ruby

ruby

Kodu kopyala

```
class Araba
  attr_accessor :marka, :yil

  def initialize(marka, yil)
    @marka = marka
    @yil = yil
  end

  def calistir
    puts "#{@marka} çalıştırıldı!"
  end
end
```

```
araba = Araba.new("Toyota", 2020)
araba.calistir
```

JavaScript

```
javascript
Kodu kopyala
class Araba {
  constructor(marka, yil) {
    this.marka = marka;
    this.yil = yil;
  }

  calistir() {
    console.log(`${this.marka} çalıştırıldı!`);
  }
}

const araba = new Araba("Toyota", 2020);
araba.calistir();
```

---

3. Kapsülleme (Encapsulation)

3.1 Teorik Açıklama

Kapsülleme, bir sınıfın özelliklerini (verilerini) ve metotlarını dış dünyadan gizleyerek güvenliği sağlar. Bu, genellikle private erişim belirteçleriyle gerçekleştirilir.

3.2 Kod Örnekleri

Java

```
java
Kodu kopyala
class Araba {
```

```
    private String marka;

    public String getMarka() {
        return marka;
    }

    public void setMarka(String marka) {
        this.marka = marka;
    }
}
```

Python

python

Kodu kopyala

```
class Araba:
    def __init__(self):
        self.__marka = ""

    @property
    def marka(self):
        return self.__marka

    @marka.setter
    def marka(self, value):
        self.__marka = value
```

C#

csharp

Kodu kopyala

```
class Araba {
```

```
    private string marka;

    public string Marka {
        get { return marka; }
        set { marka = value; }
    }
}
```

Ruby

ruby

Kodu kopyala

```
class Araba
  def initialize
    @marka = ""
  end

  def marka
    @marka
  end

  def marka=(value)
    @marka = value
  end
end
```

JavaScript

javascript

Kodu kopyala

```
class Araba {
  #marka;
```

```
  constructor() {
    this.#marka = "";
  }

  get marka() {
    return this.#marka;
  }

  set marka(value) {
    this.#marka = value;
  }
}
```

---

4. Kalıtım (Inheritance)

4.1 Teorik Açıklama

Kalıtım, bir sınıfın (alt sınıf) başka bir sınıfın (üst sınıf) özelliklerini ve metotlarını devralmasıdır. Bu, kodun yeniden kullanılabilirliğini artırır.

4.2 Kod Örnekleri

Java

java

Kodu kopyala

```
class Arac {
    void hareketEt() {
        System.out.println("Araç hareket ediyor.");
    }
}
```

```
class Araba extends Arac {
  void calistir() {
    System.out.println("Araba çalıştı!");
  }
}
```

Python

```
python
Kodu kopyala
class Arac:
  def hareket_et(self):
    print("Araç hareket ediyor.")

class Araba(Arac):
  def calistir(self):
    print("Araba çalıştı!")
```

C#

```
csharp
Kodu kopyala
class Arac {
  public void HareketEt() {
    Console.WriteLine("Araç hareket ediyor.");
  }
}

class Araba : Arac {
  public void Calistir() {
    Console.WriteLine("Araba çalıştı!");
  }
```

```
}
```

Ruby

```
ruby
Kodu kopyala
class Arac
  def hareket_et
    puts "Araç hareket ediyor."
  end
end

class Araba < Arac
  def calistir
    puts "Araba çalıştı!"
  end
end
```

JavaScript

```
javascript
Kodu kopyala
class Arac {
  hareketEt() {
    console.log("Araç hareket ediyor.");
  }
}

class Araba extends Arac {
  calistir() {
    console.log("Araba çalıştı!");
  }
```

}

---

5. Polimorfizm (Polymorphism)

5.1 Teorik Açıklama

Polimorfizm, aynı isimli metotların farklı sınıflarda farklı davranışlar sergilemesini sağlar.

5.2 Kod Örnekleri

Java

java

Kodu kopyala

```
class Arac {
  void hareketEt() {
    System.out.println("Araç hareket ediyor.");
  }
}

class Araba extends Arac {
  @Override
  void hareketEt() {
    System.out.println("Araba yolda gidiyor.");
  }
}
```

Python

python

Kodu kopyala

```
class Arac:
  def hareket_et(self):
    print("Araç hareket ediyor.")
```

```
class Araba(Arac):
    def hareket_et(self):
        print("Araba yolda gidiyor.")
```

Bu şekilde devam eden kapsamlı bir not hazırladım. İsterseniz eklemek istediğiniz başka başlık veya örnek varsa belirtebilirsiniz! 😊

4o

Şunu dedin:

biraz daha detaylandırırmısın

ChatGPT:

ChatGPT

Tabii ki! Eğitim notunu daha kapsamlı hale getirerek her kavramı daha detaylı açıklamalar ve kod örnekleri ile destekleyeceğim. İşte genişletilmiş hali:

Nesne Yönelimli Programlama (OOP) Eğitim Notları

1. Nesne Yönelimli Programlamaya Giriş

Nesne Yönelimli Programlama (Object-Oriented Programming - OOP), yazılımın modüler ve yeniden kullanılabilir olmasını sağlayan bir yöntemdir. Bu yaklaşım, yazılımı gerçek dünya varlıklarını modelleyecek şekilde organize eder.

1.1 Temel Kavramlar

1.1.1 Sınıflar ve Nesneler

- **Sınıf (Class):** Nesnelerin şablonu veya planıdır.
- **Nesne (Object):** Sınıfın bir örneğidir ve bellekte oluşturulur.

1.1.2 Kapsülleme (Encapsulation)

- Veriler ve metotlar bir sınıf içinde saklanır.
- **Amaç:** Verilere yalnızca kontrollü erişim sağlamak.

1.1.3 Kalıtım (Inheritance)

- Bir sınıfın özelliklerini başka bir sınıfa devretmesidir.
- **Amaç:** Kod tekrarını azaltmak.

1.1.4 Polimorfizm (Polymorphism)

- Aynı isimli metotların farklı sınıflarda farklı şekilde davranmasını sağlar.
- **Amaç:** Dinamik ve esnek bir yapı oluşturmak.

1.1.5 Soyutlama (Abstraction)

- Karmaşıklığı gizler ve yalnızca önemli detayları gösterir.
- **Amaç:** Kullanıcıyı gereksiz detaylardan korumak.

---

2. Sınıf ve Nesne

2.1 Teorik Açıklama

Sınıf ve nesne, OOP'nin temel yapı taşlarıdır. Bir sınıf; değişkenler (özellikler) ve metotlar (işlevler) içerir. Nesne ise bu sınıfın bellekteki örneğidir.

2.2 Kod Örnekleri

Java

java

Kodu kopyala

```
class Araba {
   String marka;
   int yil;

   void calistir() {
      System.out.println(marka + " çalıştırıldı!");
   }
}

public class Main {
   public static void main(String[] args) {
      Araba araba = new Araba();
      araba.marka = "Toyota";
```

```
        araba.yil = 2020;
        araba.calistir();
    }
}
```

Python

```
python
Kodu kopyala
class Araba:
    def __init__(self, marka, yil):
        self.marka = marka
        self.yil = yil

    def calistir(self):
        print(f"{self.marka} çalıştırıldı!")

araba = Araba("Toyota", 2020)
araba.calistir()
```

Ruby

```
ruby
Kodu kopyala
class Araba
  attr_accessor :marka, :yil

  def initialize(marka, yil)
    @marka = marka
    @yil = yil
  end
```

```
 def calistir
  puts "#{@marka} çalıştırıldı!"
 end
end

araba = Araba.new("Toyota", 2020)
araba.calistir
```

---

3. Kapsülleme (Encapsulation)

3.1 Teorik Açıklama

Kapsülleme, bir sınıfın özelliklerini (verilerini) ve metotlarını dış dünyadan gizleyerek güvenliği sağlar. Kullanıcı, sadece izin verilen metotlar aracılığıyla verilere erişebilir.

3.2 Kod Örnekleri

Java

```
java
Kodu kopyala
class Araba {
  private String marka;

  public String getMarka() {
    return marka;
  }

  public void setMarka(String marka) {
    this.marka = marka;
  }
}
```

Python

python

Kodu kopyala

```
class Araba:
    def __init__(self):
        self.__marka = ""

    @property
    def marka(self):
        return self.__marka

    @marka.setter
    def marka(self, value):
        self.__marka = value
```

C#

csharp

Kodu kopyala

```
class Araba {
    private string marka;

    public string Marka {
        get { return marka; }
        set { marka = value; }
    }
}
```

---

4. Kalıtım (Inheritance)

4.1 Teorik Açıklama

Kalıtım, bir sınıfın başka bir sınıfın özelliklerini ve metotlarını devralmasıdır. Bu özellik, kod tekrarını azaltır ve nesneler arasında bir hiyerarşi kurulmasını sağlar.

4.2 Kod Örnekleri

Java

java

Kodu kopyala

```
class Arac {
  void hareketEt() {
    System.out.println("Araç hareket ediyor.");
  }
}

class Araba extends Arac {
  void calistir() {
    System.out.println("Araba çalıştı!");
  }
}
```

Python

python

Kodu kopyala

```
class Arac:
  def hareket_et(self):
    print("Araç hareket ediyor.")

class Araba(Arac):
  def calistir(self):
    print("Araba çalıştı!")
```

Ruby

ruby

Kodu kopyala

```
class Arac
  def hareket_et
    puts "Araç hareket ediyor."
  end
end


class Araba < Arac
  def calistir
    puts "Araba çalıştı!"
  end
end
```

---

5. Polimorfizm (Polymorphism)

5.1 Teorik Açıklama

Polimorfizm, aynı isimli metotların farklı sınıflarda farklı şekillerde çalışmasını sağlar. Örneğin bir hareketEt metodu, Araba ve Bisiklet sınıflarında farklı şekilde davranabilir.

5.2 Kod Örnekleri

Java

java

Kodu kopyala

```
class Arac {
    void hareketEt() {
        System.out.println("Araç hareket ediyor.");
    }
}
```

```
class Araba extends Arac {
  @Override
  void hareketEt() {
    System.out.println("Araba yolda gidiyor.");
  }
}
```

Python

python

Kodu kopyala

```
class Arac:
  def hareket_et(self):
    print("Araç hareket ediyor.")

class Araba(Arac):
  def hareket_et(self):
    print("Araba yolda gidiyor.")
```

C#

csharp

Kodu kopyala

```
class Arac {
  public virtual void HareketEt() {
    Console.WriteLine("Araç hareket ediyor.");
  }
}

class Araba : Arac {
  public override void HareketEt() {
    Console.WriteLine("Araba yolda gidiyor.");
```

```
    }
}
```

---

6. Proje: Kütüphane Yönetim Sistemi

6.1 Tanım

Bir kütüphane yönetim sistemi tasarlıyoruz. Bu sistemde:

- **Kitap** sınıfı: Kitap bilgilerini tutar.
- **Kullanıcı** sınıfı: Kullanıcı bilgilerini ve ödünç alma işlemlerini yönetir.
- **Yönetici** sınıfı: Kitap ekleme ve yönetme işlemlerini gerçekleştirir.

6.2 Python'da Örnek Kod

python

Kodu kopyala

```
class Kitap:
    def __init__(self, isim, yazar):
        self.isim = isim
        self.yazar = yazar

    def __str__(self):
        return f"{self.isim} - {self.yazar}"

class Kullanici:
    def __init__(self, ad):
        self.ad = ad
        self.odunc_aldiklari = []

    def kitap_odunc_al(self, kitap):
        self.odunc_aldiklari.append(kitap)
        print(f"{self.ad}, '{kitap.isim}' kitabını ödünç aldı.")
```

```
# Kullanım
kitap1 = Kitap("Simyacı", "Paulo Coelho")
kullanici = Kullanici("Ahmet")
kullanici.kitap_odunc_al(kitap1)
```